# Lab EKS — Console AWS

À la fin de ce lab, nous aurons :

- une VPC ;
- deux subnets publics ;
- deux subnets privés ;
- un Internet Gateway ;
- une NAT Gateway ;
- des route tables ;
- un cluster EKS ;
- un Managed Node Group ;
- des nodes `Ready` ;
- une application Nginx exposée via Load Balancer.

## Architecture

```
Internet
   |
Internet Gateway
   |
Subnets publics
   |
NAT Gateway
   |
Subnets privés
   |
Nodes EKS
```

> Région utilisée dans les exemples : **us-east-1**. Remplace-la par ta région. Utilise la même région partout.

---

# Étape 1 — Préparer le compte AWS

## 1.1 Choisir la région

Dans la console AWS, en haut à droite, choisis une région qui supporte EKS.

Exemple :

```
United States (N. Virginia) us-east-1
```

Ne change pas de région pendant le lab.

## 1.2 Vérifier les quotas

Dans **Service Quotas**, vérifie que tu peux créer :

- des instances EC2 ;
- des Elastic IP ;
- une NAT Gateway.

La NAT Gateway peut générer des coûts. Supprime-la à la fin du lab.

---

# Étape 2 — Créer la VPC

## 2.1 Ouvrir la console VPC

1. Cherche **VPC** dans la console AWS.
2. Ouvre **VPC**.
3. Dans le menu gauche, clique sur **Your VPCs**.
4. Clique sur **Create VPC**.

## 2.2 Choisir VPC and more

Dans **Resources to create**, sélectionne :

```
VPC and more
```

Remplis les champs ainsi :

| Champ | Valeur |
|---|---|
| Name tag | `eks-lab` |
| IPv4 CIDR block | `10.0.0.0/16` |
| IPv6 CIDR block | No IPv6 CIDR block |
| Tenancy | Default |
| Number of Availability Zones | 2 |
| Number of public subnets | 2 |
| Number of private subnets | 2 |
| NAT gateways | In 1 AZ pour ce lab |
| VPC endpoints | None |
| DNS hostnames | Activé |
| DNS resolution | Activé |

Pour un lab moins cher, choisis :

```
NAT gateways: 1
```

Puis clique sur **Create VPC**.

## 2.3 Ce que AWS crée automatiquement

AWS doit créer :

- `eks-lab` ;
- 2 subnets publics ;
- 2 subnets privés ;
- 1 Internet Gateway ;
- 1 NAT Gateway ;

- 1 Elastic IP ;
- une route table publique ;
- une route table privée ;
- les routes nécessaires.

Attends que la NAT Gateway soit à l'état :

```
Available
```

Ne continue pas tant qu'elle est `Pending`.

---

# Étape 3 — Vérifier la VPC

## 3.1 Vérifier DNS

Dans **VPC** → **Your VPCs** :

1. Sélectionne `eks-lab`.
2. Clique sur **Actions**.
3. Choisis **Edit VPC settings**.
4. Vérifie :

```
Enable DNS resolution: activé
Enable DNS hostnames: activé
```

Ces deux options sont importantes. Sans DNS, les nodes peuvent ne pas rejoindre le cluster.

## 3.2 Identifier les subnets

Dans **VPC** → **Subnets**, tu dois voir 4 subnets.

Ajoute ces tags aux subnets publics :

```
kubernetes.io/role/elb = 1
kubernetes.io/cluster/eks-lab = shared
```

Ajoute ces tags aux subnets privés :

```
kubernetes.io/role/internal-elb = 1
kubernetes.io/cluster/eks-lab = shared
```

Remplace `eks-lab` par le vrai nom de ton cluster si tu utilises un autre nom.

## 3.3 Vérifier les route tables

Dans **VPC** → **Route tables**, identifie la route table publique.

Elle doit avoir :

```
Destination: 0.0.0.0/0
Target: Internet Gateway igw-xxxxx
```

Elle doit être associée aux deux subnets publics.

Identifie ensuite la route table privée.

Elle doit avoir :

```
Destination: 0.0.0.0/0
Target: NAT Gateway nat-xxxxx
```

Elle doit être associée aux deux subnets privés.

## 3.4 Vérifier l'Internet Gateway

Dans **VPC** → **Internet Gateways** :

1. Sélectionne l'Internet Gateway.
2. Vérifie qu'elle est attachée à `eks-lab`.

Si elle n'est pas attachée, les subnets publics ne fonctionneront pas.

## 3.5 Vérifier la NAT Gateway

Dans **VPC → NAT Gateways** :

1. Ouvre la NAT Gateway.
2. Vérifie :

```
State: Available
Subnet: subnet public
Elastic IP: présente
```

Une NAT Gateway dans un subnet privé est une erreur.

---

# Étape 4 — Créer le rôle IAM du cluster

## 4.1 Ouvrir IAM

1. Cherche **IAM**.
2. Ouvre **IAM → Roles**.
3. Clique sur **Create role**.

## 4.2 Configurer le rôle

Choisis :

```
Trusted entity type: AWS service
Use case: EKS - Cluster
```

Clique sur **Next**.

Ajoute la policy AWS suivante :

```
AmazonEKSClusterPolicy
```

Nom du rôle :

```
eks-lab-cluster-role
```

Clique sur **Create role**.

> Ne réutilise pas ce rôle pour les nodes. Le cluster role et le node role doivent être différents.

---

# Étape 5 — Créer le cluster EKS

## 5.1 Ouvrir EKS

1. Cherche **Elastic Kubernetes Service**.
2. Ouvre **EKS**.
3. Clique sur **Clusters**.
4. Clique sur **Create cluster**.

## 5.2 Configuration générale

Pour ce lab, choisis la configuration classique, sans EKS Auto Mode.

Remplis :

| Champ | Valeur |
|---|---|
| Name | `eks-lab` |
| Kubernetes version | Version proposée par AWS |
| Cluster service role | `eks-lab-cluster-role` |
| EKS Auto Mode | Désactivé pour ce lab |

Ne choisis pas manuellement une ancienne version Kubernetes. Utilise une version actuellement proposée par AWS et disponible dans ta région.

## 5.3 Configurer le réseau du cluster

Dans **Specify networking** :

| Champ | Valeur |
| --- | --- |
| VPC | `eks-lab` |
| Subnets | Les 2 subnets privés |
| Security groups | Laisser le cluster SG créé par EKS |
| IP family | IPv4 |
| Cluster endpoint access | Public and private, si disponible |

Pour un premier lab, le plus simple est :

```
Public access: activé
Private access: activé si disponible
```

Si tu choisis seulement `Private`, ton ordinateur doit avoir un accès réseau à la VPC. Sinon `kubectl` ne fonctionnera pas depuis ton ordinateur.

Pour la partie CIDR de l'endpoint public, tu peux laisser la valeur par défaut pour un lab. En production, limite l'accès à ton IP ou à ton réseau.

## 5.4 Add-ons

Garde les add-ons recommandés par AWS :

- Amazon VPC CNI ;
- CoreDNS ;
- kube-proxy.

Choisis les versions proposées par AWS.

Clique sur **Next** jusqu'à la page de résumé, puis clique sur **Create**.

## 5.5 Attendre le statut Active

Dans **EKS** → **Clusters**, attends :

```
Status: Active
```

Ne crée pas le node group avant que le cluster soit `Active`.

# Étape 6 — Créer le rôle IAM des nodes

## 6.1 Créer le rôle

Dans **IAM** → **Roles** → **Create role** :

```
Trusted entity type: AWS service
Use case: EC2
```

Clique sur **Next**.

## 6.2 Ajouter les policies

Pour un lab simple, ajoute ces trois policies :

```
AmazonEKSWorkerNodePolicy
AmazonEC2ContainerRegistryReadOnly
AmazonEKS_CNI_Policy
```

AWS recommande dans certaines architectures avancées de donner la policy CNI à un rôle séparé associé au service account `aws-node`. Pour ce lab débutant, la laisser sur le node role est plus simple.

Nom du rôle :

```
eks-lab-node-role
```

Clique sur **Create role**.

## 6.3 Vérifier le rôle

Dans le rôle `eks-lab-node-role`, vérifie que les trois policies sont réellement dans l'onglet **Permissions**.

Vérifie aussi que le rôle n'a pas :

- de permissions boundary restrictive ;
- de policy SCP de compte qui bloque EC2, EKS, ECR ou STS.

---

# Étape 7 — Créer le Managed Node Group

## 7.1 Ouvrir la page Compute

1. Dans EKS, ouvre le cluster `eks-lab`.
2. Clique sur **Compute**.
3. Clique sur **Add node group**.

## 7.2 Nom et rôle

Remplis :

| Champ | Valeur |
|---|---|
| Name | `eks-lab-nodes` |
| Node IAM role | `eks-lab-node-role` |

Ne sélectionne pas `eks-lab-cluster-role`.

## 7.3 Configuration de calcul

Choisis :

| Champ | Valeur recommandée pour le lab |
|---|---|
| AMI type | Amazon Linux 2023 ×86_64 Standard, si proposé |
| Capacity type | On-Demand |
| Instance type | `t3.small` |
| Disk size | 20 GiB |
| SSH access | Désactivé pour ce lab |
| Launch template | Aucun |

Si `t3.small` n'est pas disponible, choisis un type x86_64 équivalent disponible dans ta région.

N'utilise pas de custom AMI et ne crée pas de launch template pour ce premier lab.

## 7.4 Configurer le scaling

Utilise :

```
Minimum size: 1
Desired size: 2
Maximum size: 2
```

Deux nodes sont utiles pour vérifier les deux Availability Zones.

## 7.5 Choisir les subnets

Sélectionne uniquement :

```
eks-private-a
eks-private-b
```

Ne sélectionne pas les subnets publics pour ce lab.

Les subnets privés doivent utiliser une route vers la NAT Gateway.

## 7.6 Autres options

Laisse les valeurs par défaut pour :

- labels ;
- taints ;
- update configuration ;
- node repair ;
- tags supplémentaires.

Clique sur **Next**, vérifie le résumé, puis clique sur **Create**.

## 7.7 Attendre Active

Attends que le node group soit :

```
Active
```

La création peut prendre plusieurs minutes.

---

# Étape 8 — Configurer kubectl

Sur ton ordinateur, exécute :

```
aws configure
```

Mets la même région que la console :

```
us-east-1
```

Vérifie ton identité :

```
aws sts get-caller-identity
```

Configure `kubectl` :

```
aws eks update-kubeconfig \
  --region us-east-1 \
  --name eks-lab
```

Teste :

```
kubectl config current-context
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -A
```

Résultat attendu :

```
STATUS: Ready
```

Les pods système doivent être principalement en :

```
Running
```

---

# Étape 9 — Déployer la première application

Maintenant que le cluster fonctionne, nous allons déployer une petite application Nginx.

Crée le fichier `nginx-deployment.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```

```
        ports:
          - containerPort: 80
```

Applique le fichier :

```
kubectl apply -f nginx-deployment.yaml
```

Vérifie :

```
kubectl get deployments
kubectl get pods
```

Nous devons avoir 3 Pods Nginx.

---

# Étape 10 — Exposer l'application avec un Load Balancer

Nos Pods fonctionnent, mais ils ne sont pas encore accessibles depuis Internet.

Nous allons donc créer un Service de type LoadBalancer.

Crée le fichier `nginx-service.yaml` :

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer
```

Puis :

```
kubectl apply -f nginx-service.yaml
```

Vérifie :

```
kubectl get svc
```

AWS va créer automatiquement un Load Balancer.

## Accéder à l'application

Pour accéder à l'application depuis ton navigateur, utilise l'adresse du **Load Balancer**, pas celle du cluster EKS.

Récupère l'URL avec :

```
kubectl get svc nginx-service
```

Regarde la colonne `EXTERNAL-IP` (ou le hostname). Elle ressemble à :

```
xxxx-xxxx.us-east-1.elb.amazonaws.com
```

Ouvre ensuite cette adresse dans ton navigateur, souvent en **HTTP** :

```
<http://xxxx-xxxx.us-east-1.elb.amazonaws.com>
```

> Attention : une URL du type `https://xxxxxxxx.gr7.us-east-1.eks.amazonaws.com/` est l'**endpoint API du cluster EKS** (utilisé par `kubectl`). Ce n'est **pas** l'URL de ton application Nginx.

Si `EXTERNAL-IP` reste à `<pending>` :

```
kubectl get pods
kubectl describe svc nginx-service
```

Causes fréquentes dans ce lab :

1. tags manquants sur les subnets publics (`kubernetes.io/role/elb = 1`) ;
2. nodes pas encore `Ready` ou pods Nginx pas `Running` ;
3. délai de quelques minutes après `kubectl apply`.

---

# Étape 11 — Tester depuis l'intérieur du cluster

Nous pouvons également vérifier que notre service fonctionne depuis Kubernetes.

Crée un petit Pod temporaire :

```
kubectl run test-pod \
  --image=busybox \
  -it \
  --rm \
  --restart=Never \
  -- sh
```

Depuis le Pod :

```
wget -O- <http://nginx-service>
```

Nous devons recevoir la réponse de Nginx.

Puis :

```
exit
```

---

# Étape 12 — Explorer les logs

Maintenant que notre application fonctionne, regardons rapidement les logs.

Pour un Pod :

```
kubectl logs <nom-du-pod>
```

Pour le Deployment :

```
kubectl logs deployment/nginx-deployment
```

Et pour suivre les logs en temps réel :

```
kubectl logs -f deployment/nginx-deployment
```

Nous pouvons également regarder les événements du cluster :

```
kubectl get events
```

---

# Étape 13 — Notions à connaître

Nous allons rencontrer plusieurs notions importantes dans la suite du cours.

## RBAC

Le RBAC permet de contrôler qui peut faire quoi dans Kubernetes.

On peut par exemple donner à une équipe des droits limités sur certains namespaces.

## Persistance

Par défaut, les données stockées directement dans un Pod ne sont pas faites pour être conservées durablement.

Pour les données persistantes, Kubernetes utilise notamment :

- PersistentVolume
- PersistentVolumeClaim
- StorageClass

Nous reviendrons sur ces notions dans un prochain lab.

---

# Étape 14 — Supprimer le lab

Supprime dans cet ordre :

1. le Managed Node Group ;
2. le cluster EKS ;
3. les éventuels Load Balancers ;
4. la NAT Gateway ;
5. l'Elastic IP ;
6. la VPC ;
7. les rôles IAM de test.

Dans la console, vérifie aussi :

- EC2 Instances ;
- Auto Scaling Groups ;
- EBS Volumes ;
- NAT Gateways ;
- Elastic IPs ;
- Load Balancers.